

Operating Systems

Project Checkpoint 2

112070002 張博安

Table of Contents

1	Screenshots for Compilation	2
2	Screenshots and Explanation	2
2.1	ThreadCreate	2
2.2	Producer	4
2.3	Consumer	5
2.4	Interrupt	6
2.5	Final Result	7

1 Screenshots for Compilation

```
boanz@BENSONCH /cygdrive/d/os/ppc2
$ make clean
rm *.hex *.ihx *.lnk *.lst *.map *.mem *.rel *.rst *.sym *.asm *.lk
rm: cannot remove '*.ihx': No such file or directory
rm: cannot remove '*.lnk': No such file or directory
make: *** [Makefile:25: clean] Error 1

boanz@BENSONCH /cygdrive/d/os/ppc2
$ make
sdcc -c testpreempt.c
testpreempt.c:50: warning 158: overflow in implicit constant conversion
sdcc -c preemptive.c
preemptive.c:203: warning 85: in function ThreadCreate unreferenced function argument : 'fp'
preemptive.c:261: warning 158: overflow in implicit constant conversion
sdcc -o testpreempt.hex testpreempt.rel preemptive.rel
```

Figure 1: Compilation result of `make clean` and `make`

The `make clean` output shows `No such file or directory` messages and `Error 1` because no compiled files exist yet. This is expected behavior and not an actual error.

The `make` output shows that SDCC successfully compiles `testpreempt.c` and `preemptive.c`, generating `testpreempt.hex`. Three non-fatal warnings appear during compilation. First, `testpreempt.c` line 50 produces an overflow warning for the implicit constant conversion of `TH1 = -6`, which is the standard 8051 baud rate convention. Second, `preemptive.c` line 203 reports that the function argument `fp` in `ThreadCreate()` is unreferenced, because `fp` is accessed through `DPTR` in inline assembly rather than directly in C. Third, `preemptive.c` line 261 produces another overflow warning for the implicit constant conversion of `-1` in `ThreadExit()`. None of these affect program execution.

2 Screenshots and Explanation

2.1 ThreadCreate

There are two `ThreadCreate` calls in total: the first in `Bootstrap` creates a thread for `main`, and the second in `main` creates a thread for `Producer`. Since `main` calls `Consumer` directly, no additional `ThreadCreate` is needed for `Consumer`. The address of each function can be found in the `.map` file; `ThreadCreate` is located at `0x016E`.

	Value	Global	Global Defined In Module
C:	00000081	_Producer	testpreempt
C:	000000AA	_Consumer	testpreempt
C:	000000DA	_main	testpreempt
C:	000000E6	__sdcc_gsinit_startup	testpreempt
C:	000000EA	__mcs51_genRAMCLEAR	testpreempt
C:	000000EB	__mcs51_genXINIT	testpreempt
C:	000000EC	__mcs51_genXRAMCLEAR	testpreempt
C:	000000ED	_timer0_ISR	testpreempt
C:	000000F3	_Bootstrap	preemptive
C:	00000119	_myTimer0Handler	preemptive
C:	0000016E	_ThreadCreate	preemptive
C:	000001E5	_ThreadYield	preemptive
C:	00000240	_ThreadExit	preemptive
C:	0000029D	_moduint	_moduint
C:	000002EA	_modsint	_modsint

Figure 2: Function address list from `testpreempt.map`

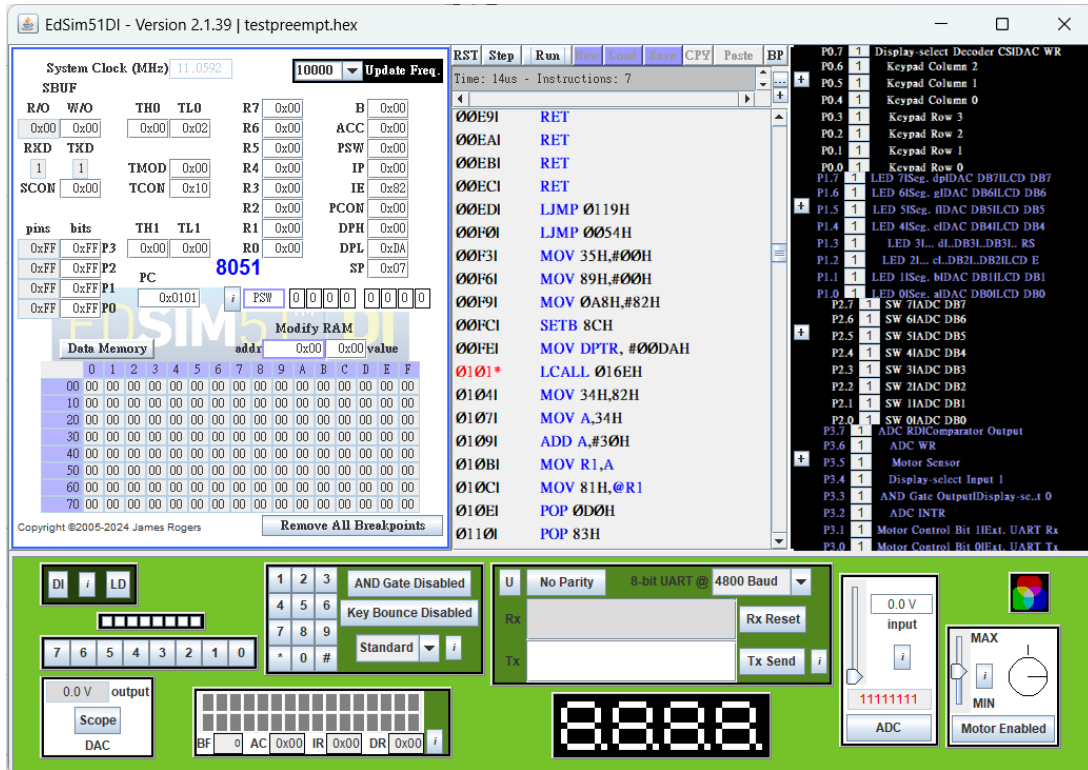


Figure 3: Screenshot before first ThreadCreate function call

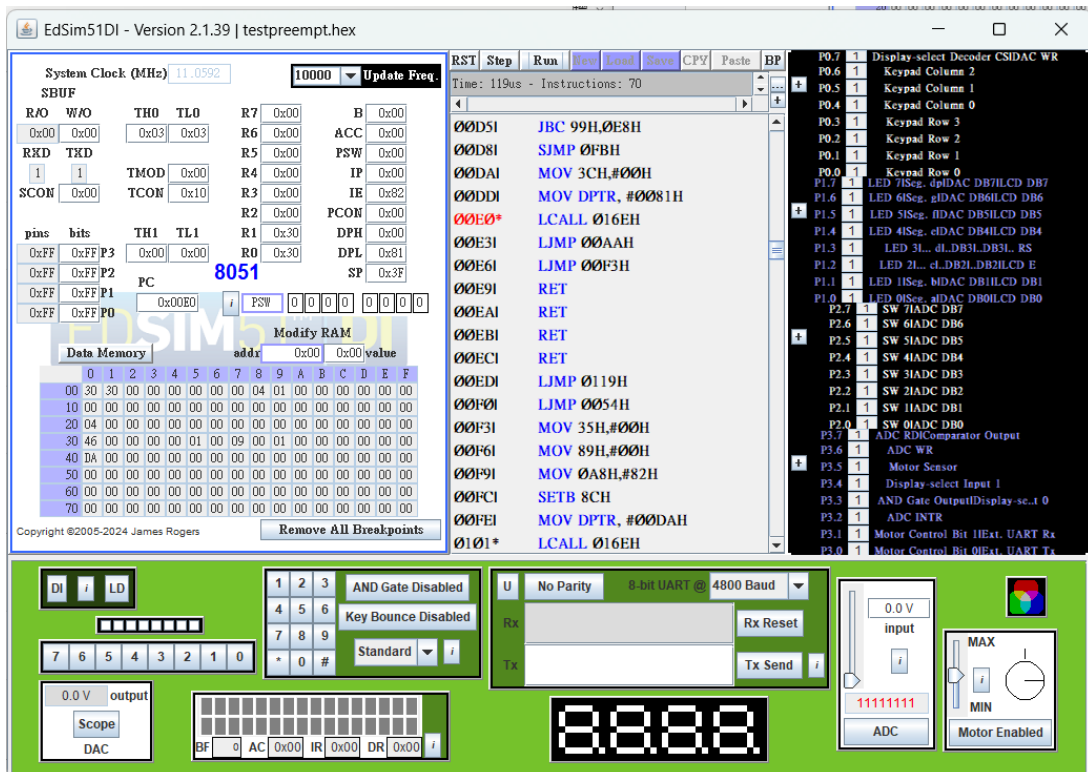


Figure 4: Screenshot before second ThreadCreate function call

Before the first ThreadCreate (PC = 0x0101): PC is stopped at 0x0101, just before Bootstrap executes LCALL 016EH. SP = 0x07, Data Memory is all zeros, and the

system is in its initial state. Timer 0 is already running (IE = 0x82), and threadMask at 0x35 = 0x00 indicates no threads have been created yet.

Before the second ThreadCreate (PC = 0x00E0): PC is stopped at 0x00E0, just before `main` executes `LCALL 016EH`. `RESTORESTATE` has already switched SP to 0x3F (Thread 0's stack). In Data Memory around 0x40, the initial stack frame for the `main` thread is visible: DPL/DPH hold `main`'s function address (0xDA/0x00), followed by zeroed register save slots and PSW. threadMask at 0x35 = 0x01 indicates Thread 0 (`main`) has been created.

Stack change explanation: `ThreadCreate` initialises a new thread's stack starting at $0x3F + newThread \times 0x10$. For Thread 0 (`main`), the base is 0x3F. The following are pushed in order:

1. DPL (low byte of `main`'s address: 0xDA)
2. DPH (high byte: 0x00)
3. ACC, B, DPL, DPH (all initialised to 0x00)
4. PSW = 0x00 (Thread 0 uses Register Bank 0)

`savedSP[0]` records the top of this stack for later use by `RESTORESTATE`. Thread 1 (`Producer`) follows the same process with base address 0x4F and PSW = 0x08 (Bank 1).

2.2 Producer

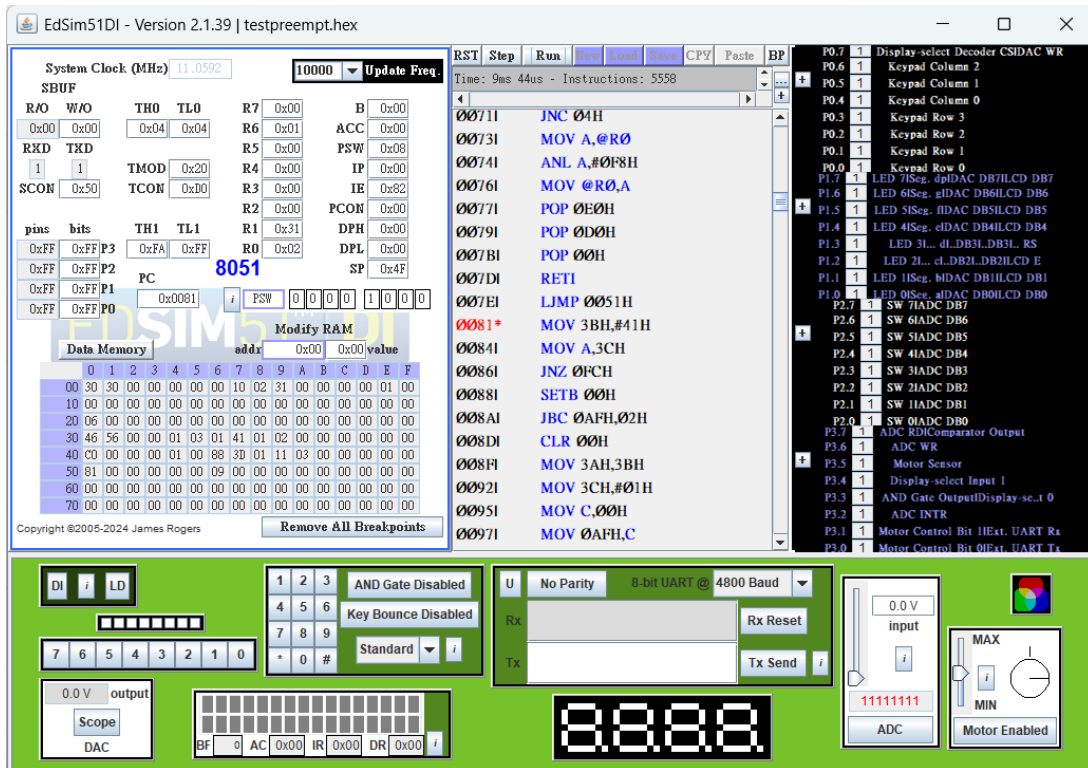


Figure 5: Screenshot for first `Producer` call

The **Producer**'s code address range is 0x0081 to 0x00A9. The screenshot shows PC stopped at 0x0081 (the first instruction of **Producer**: MOV 3BH, #41H), confirming that **Producer** is running.

We can tell **Producer** is running because:

- PC is within the range 0x0081–0x00A9
- PSW = 0x08, RS1:RS0 = 01, indicating the CPU is using Register Bank 1, which was assigned to Thread 1 (**Producer**) by **ThreadCreate**
- SP = 0x4F, within Thread 1's stack region (0x4F–0x5F)

2.3 Consumer

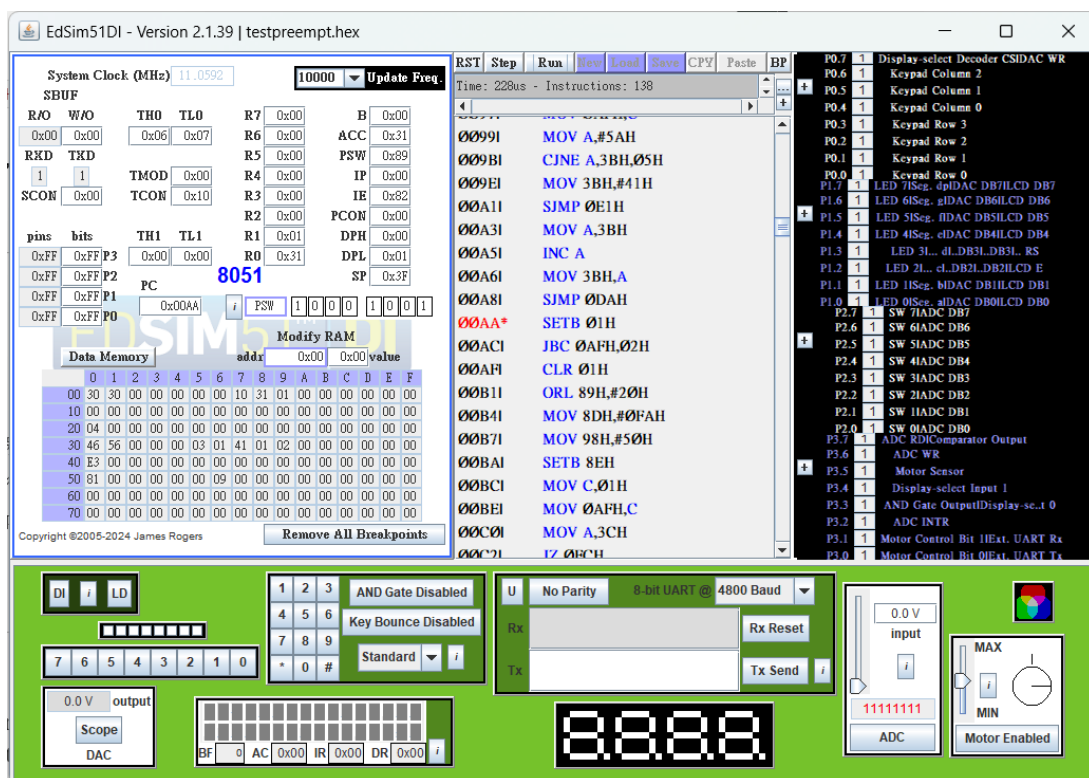


Figure 6: Screenshot for first **Consumer** call

The **Consumer**'s code address range is 0x00AA to 0x00D9. The screenshot shows PC stopped at 0x00AA (the first instruction of **Consumer**: SETB 01H), confirming that **Consumer** is running.

We can tell **Consumer** is running because:

- PC is within the range 0x00AA–0x00D9
- SP = 0x3F, within Thread 0's stack region, since **Consumer** is called directly by **main** and runs in Thread 0's context
- The Rx window shows characters being transmitted, confirming **Consumer** has successfully written to SBUF

2.4 Interrupt

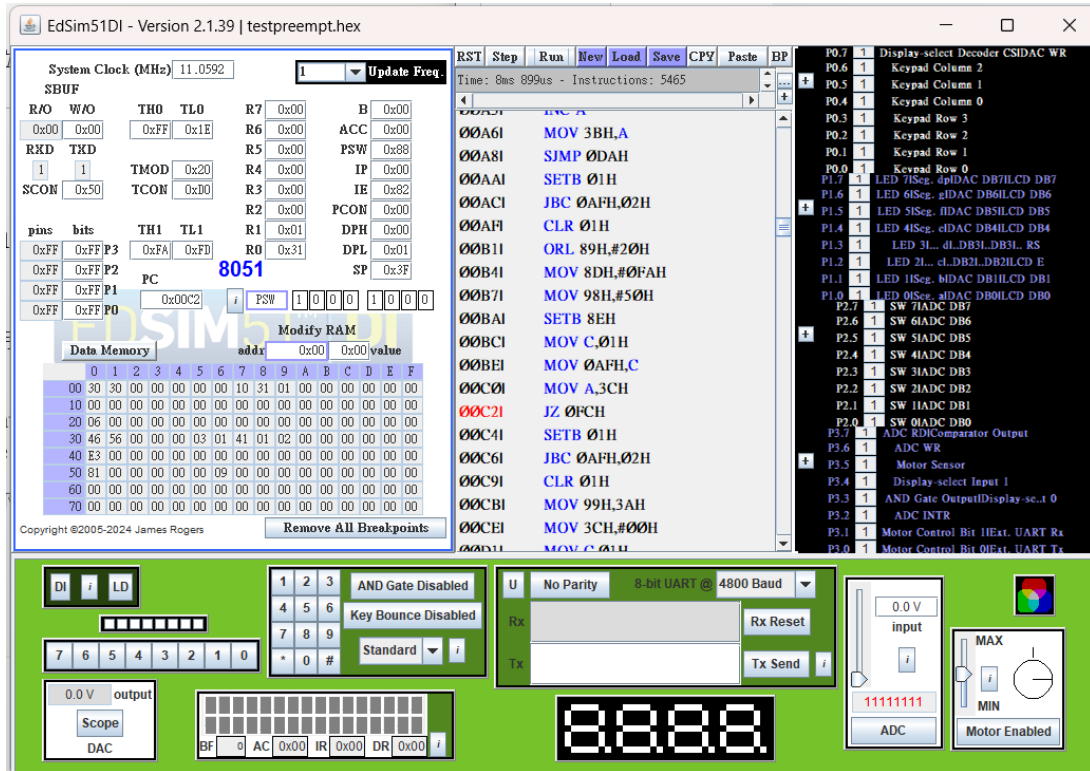


Figure 7: Screenshot before first **Producer** call was preempted

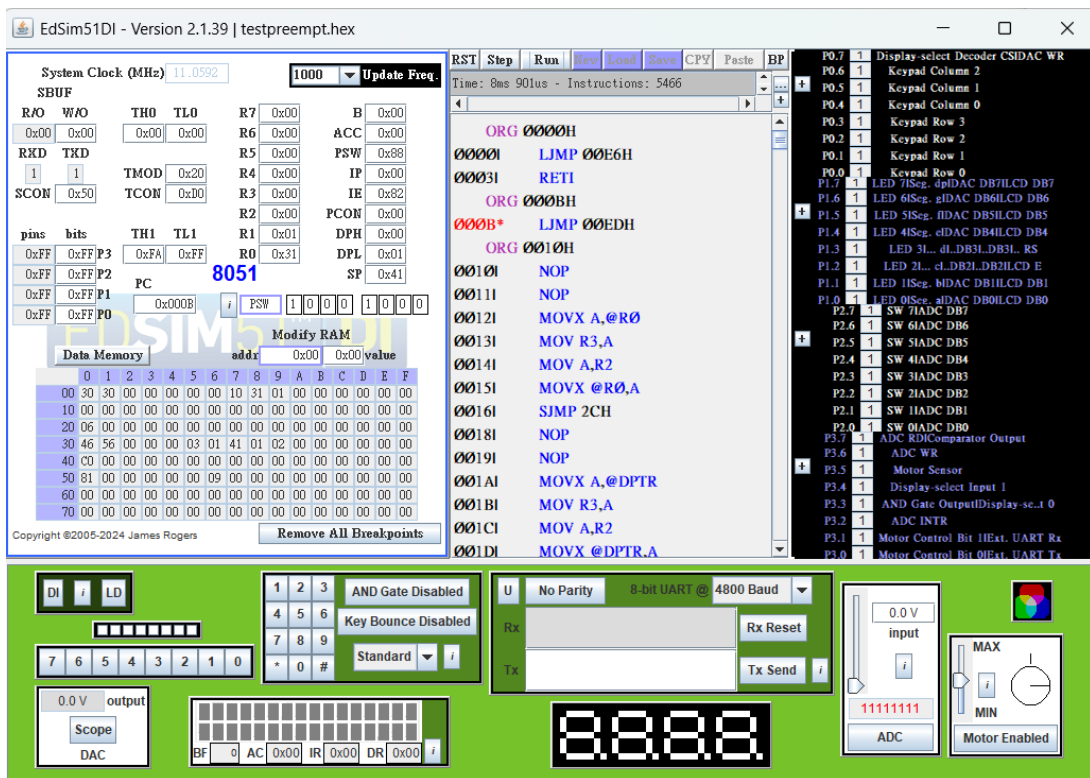


Figure 8: Screenshot after first **Producer** call was preempted

The program uses Timer 0 in mode 0 (13-bit timer) to generate periodic preemption interrupts. Timer 0 counts continuously and triggers an interrupt when it overflows from 0x1FFF to 0x0000, causing PC to jump to the interrupt vector at 0x000B.

Before the interrupt: PC is stopped at 0x00C2 (inside [Consumer](#)'s waiting loop). TH0 = 0xFF and TL0 = 0x1E, showing Timer 0 is about to overflow. PSW = 0x88 indicates the CPU is in Bank 1, meaning the interrupt will fire during [Producer](#)'s execution.

After the interrupt: After Timer 0 overflows, PC jumps to 0x000B (LJMP 00EDH), then to [timer0_ISR](#) at 0x00ED, and finally to [myTimer0Handler](#) at 0x0119 via LJMP 0119H. TH0 and TL0 have reset to 0x00, confirming the interrupt just fired. [myTimer0Handler](#) selects the next live thread using Round-Robin scheduling, performs context switching via [SAVESTATE/RESTORESTATE](#), and returns with RETI as required for an interrupt service routine.

2.5 Final Result

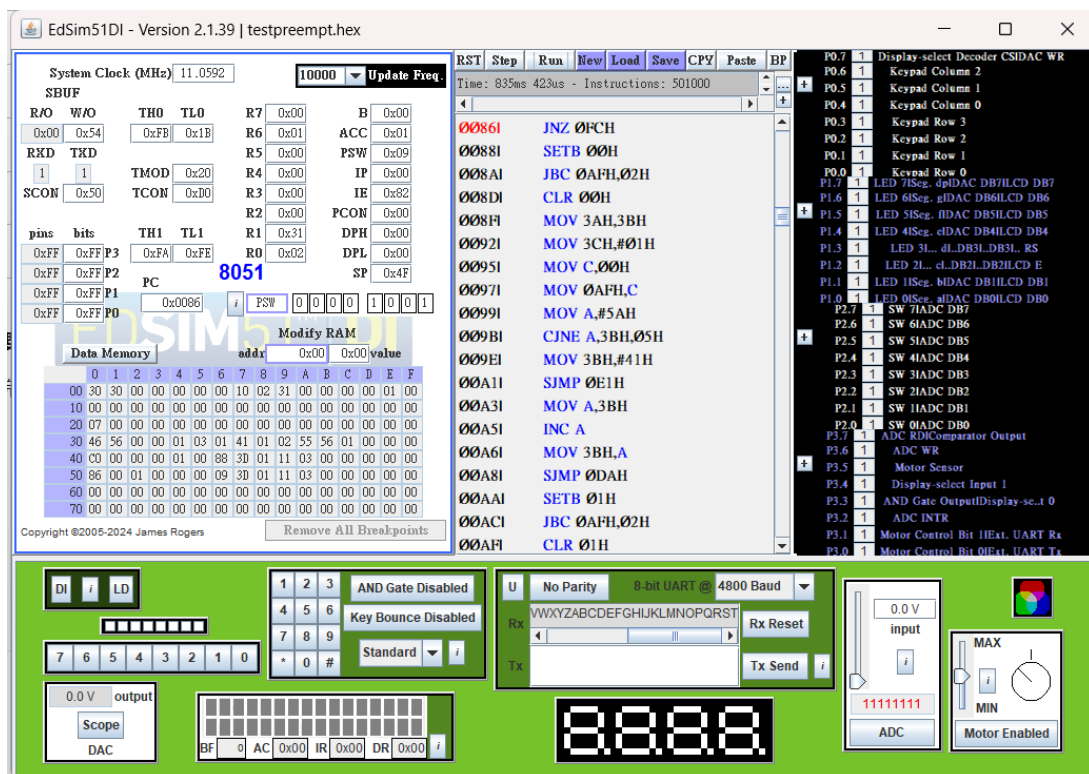


Figure 9: Screenshot after executing for some time

After running for some time, the Rx window continuously displays WXYZABCDEF... with characters correctly cycling from Z back to A. `bufferReady` at 0x3C = 0x01 indicates the buffer currently holds valid data waiting to be consumed by [Consumer](#). The overall result is correct and preemptive scheduling is functioning as intended.